ЛАБОРАТОРНА РОБОТА №9
ВИКОРИСТАННЯ ФУНКЦІЙ ДОСТУПУ ДО ОПЕРАЦІЙНОЇ СИСТЕМИ

Мета роботи: отримати навички програмування на мові Python з використанням доступу до операційної системи.

3.1 Теоретичні відомості

Робота з файловою системою передбачає створення, перейменування, переміщення або звернення до файлів і каталогів. Усі ці та багато інших завдань можна виконати за допомогою програм, написаних на Python.

Python надає багато системних функцій, що містяться в модулі **os** (скорочення від Operating System - операційна система), який необхідно імпортувати для роботи програм.

Частина назви файла, яка слідує за останньою крапкою, називається **розширенням файла** і вказує на **тип файла**. Розширення файла, зазвичай, надає програма, у якій він був створений.

Наприклад, файл з назвою **projects.docs** має тип файла - документ Word, про що свідчить розширення .docs. Частина назви файла (projects) до останньої крапки - це **ім'я файла**, яке вказує користувач.

Файл має дві ключові ознаки: **назва файла** і **шлях до файла**.

Шлях визначає, де саме на комп'ютері знаходиться файл. Наприклад, файл з назвою **projects.docs** може мати шлях до файла C:\Users\gt\Documents. Це означає, що файл **projects.docs** зберігається у каталозі **Documents**.

Частина C:\ шляху до файла - це **кореневий каталог**, який містить усі інші каталоги (**gt**, **Users**) і сам файл **projects.docs**.

У системах Windows кореневим каталогом є C:\, а шляхи до файлів записують з використанням **зворотного слешу** (\), який розділяє імена каталогів. В Linux Ubuntu кореневим каталогом є каталог /, а розділення імен каталогів виконують з використанням **прямого слеша** (/).

Шлях до файла (або каталогу), який враховує використання обох слешів, можна сформувати з використанням функції **join()**, яка міститься у модулі **path**, який у свою чергу міститься у модулі **os.**

Якщо передати у цю функцію значення назв файлів і каталогів й виконати наступний код

```
import os
print(os.path.join('usr', 'bin', 'media'))
```

функція поверне значення, у якому сформується шлях з коректною підстановкою слеша-розділювача:

```
usr\bin\media
```

Якщо ж виконати даний приклад в інтерактивному режимі інтерпретатора Python, виклик функції **os.path.join()** поверне рядок:

```
'usr\\bin\\media'
```

Якщо викликати вищезгадану фукнцію **os.path.join()** в Linux Ubuntu, вона поверне рядок:

```
'usr/bin/media'
```

Отже, для формування шляхів до файлів або каталогів у вигляді рядків використовується функція **os.path.join()**. Наприклад, наступний код

```
import os
myfiles = ['accounts.txt', 'details.csv', 'invite.docs']
for filename in myfiles:
    print(os.path.join('C:\\Users\\gt\\Documents', filename))
```

формує шляхи до файлів, додаючи до імен каталогів назви файлів зі списку:

```
C:\Users\gt\Documents\accounts.txt
C:\Users\gt\Documents\details.csv
C:\Users\gt\Documents\invite.docs
```

Кожна програма, яка виконується на комп’ютері, має свій **поточний робочий каталог** (current working directory - cwd).

Будь-які шляхи до файлів або каталогів, які не починаються з кореневого каталогу, задаються відносно поточного робочого каталогу.

Для отримання значення поточного робочого каталогу у вигляді рядка використовується функція **os.getcwd()**, а для переходу з поточного робочого каталогу у інший каталог - функція **os.chdir()**.

```
import os
print(os.getcwd())
os.chdir('C:\\Windows\\Temp')
```

print(os.getcwd())

У даному випадку, в якості робочого поточного каталогу встановлюється каталог по шляху C:\Python36, відбувається перехід у інший каталог, який стає поточним:

C:\Python36

C:\Windows\Temp

Існує два способи визначення шляху до файла або каталогу:

- абсолютний шлях - завжди починається з імені кореневого каталогу;

- відносний шлях - задається відносно поточного робочого каталогу програми.

Існують також каталоги, які позначаються одним (.) або двома (..) символами крапки. Це не реальні каталоги, а спеціальні імена, які використовуються для визначення шляхів.

Для створення нових каталогів використовується функція **os.makedirs()**, яка створює усі необхідні проміжні каталоги:

os.makedirs('C:\\Python36\\other\\myprograms')

Модуль **os.path** містить функції для визначення абсолютного шляху по заданому відносному, а також для перевірки того, чи шлях є абсолютним.

Виклик **os.path.abspath(path) -** повертає рядок абсолютного шляху до файла або каталогу.

Виклик **os.path.isabs(path) -** повертає True, якщо шлях є абсолютним, і False у протилежному випадку, коли шлях виявиться відносним.

Виклик **os.path.relpath(path, start) -** повертає рядок відносного шляху від точки start до path. Якщо start не вказаний, то в якості нього використовується поточний робочий каталог.

На наступному прикладі коду:

```
import os
print(os.path.abspath('.'))
print(os.path.abspath('.\\Scripts'))
print(os.path.isabs('.'))
print(os.path.isabs(os.path.abspath('.')))
print(os.path.relpath('C:\\Windows', 'C:\\'))
print(os.path.relpath('C:\\Windows', 'C:\\Python36\\Scripts'))
print(os.getcwd())
```

робота цих функцій виглядає так:

```
C:\Python36
C:\Python36\Scripts
False
True
Windows
..\..\Windows
C:\Python36
```

Для отримання із повного шляху лише назви файла використовують функцію **os.path.basename()**. Якщо необхідно отримати частину шляху без назви файла використовують функцію **os.path.dirname()**.

Якщо шлях до файла вважати таким рядком 'C:\\Windows\\System32\\calc.exe', то ці дві функції:

```
import os
path = 'C:\\Windows\\System32\\calc.exe'
print(os.path.basename(path))
print(os.path.dirname(path))
```

відпрацюють так:

```
calc.exe
C:\Windows\System32
```

Щоб отримати одночасно і назву файла, так і імена каталогів на шляху до нього, можна скористатися функцією **split()** у такому виконанні:

```
import os
path = 'C:\\Windows\\System32\\calc.exe'
print(os.path.split(path))
```

Результатом виведення буде кортеж з двох елементів:

```
('C:\\Windows\\System32', 'calc.exe')
```

Щоб отримати список усіх каталогів на шляху до файла і назву самого файла використовують розділювач **os.path.sep**.

```
import os
path = 'C:\\Windows\\System32\\calc.exe'
print(path.split(os.path.sep))
```

Для системи Windows буде такий результат:

```
['C:', 'Windows', 'System32', 'calc.exe']
```

Для системи Linux Ubuntu

```
import os
print('/usr/bin'.split(os.path.sep))
```

першим елементом такого списку буде порожній рядок:

['', 'usr', 'bin']

У модулі **os.path** присутні функції, які дозволяють обчислювати розміри файлів у байтах і визначати, які файли та каталоги знаходяться у вказаному каталозі. До таких функцій належать:

- os.path.getsize(path) - повертає у байтах розмір файла, шлях до якого вказаний у path;

- os.listdir(path) - повертає список рядків з назвами усіх файлів, шлях до яких вказаний у path.

Коли виконати поданий код

```
import os
print(os.path.getsize('C:\\Windows\\System32\\calc.exe'))
print(os.listdir('C:\\Windows\\System32'))
```

отримаємо розмір у байтах вказаного файла і список усіх файлів вказаного каталогу.

Для знаходження сумарного розміру файлів, наприклад, у каталозі по шляху C:\\Python36, можна використати таку програму:

```
total = 0
for filename in os.listdir('C:\\Python36'):
    total = total + os.path.getsize(os.path.join('C:\\Python36', filename))
print(total)
```

Результатом виконання даної програми буде розмір у байтах усіх файлів каталогу по шляху C:\\Python36:

4582106

Функції Python для роботи з файлами і каталогами можуть повідомляти про помилки, коли не знаходять шлях до файла або каталогу. Модуль **os.path** містить функції, які перевіряють, чи існує шлях до каталогу або файла:

– **os.path.exists(path) -** повертає значення True, якщо **файл (або каталог)** за вказаним шляхом path існує, і значення False у протилежному випадку;

– **os.path.isfile(path) -** повертає значення True, якщо **файл** за вказаним шляхом path існує, і значення False у протилежному випадку;

– **os.path.isdir(path) -** повертає значення True, якщо **каталог** за вказаним шляхом path існує, і значення False у протилежному випадку.

3.2 Завдання до виконання

Завдання 1. Виконайте вправи:

1. Виведіть на екран повний шлях і список файлів поточного каталогу.
2. Виведіть на екран розмір у байтах будь-якого файла у каталозі.
3. Здійсніть пошук файлів у поточному каталозі за певним шаблоном і виведіть результати на екран.
4. У поточному каталозі вручну створіть два текстові файли, перший з яких містить довільний текст, другий - порожній. Використовуючи функції для роботи з файлами, виконайте такі дії: виконайте копіювання даних з першого файла у другий, зчитайте з другого файла скопійовані дані і виведіть їх на екран, а потім видаліть перший файл.
5. Виведіть на екран поточну дату й час.
6. Виведіть на екран кількість днів і років, які пройшли з початку «епохи Unix». Вважати 1 рік = 365 днів.

Завдання 2. Напишіть програми у середовищі програмування для розв'язування таких завдань:

1. Запишіть програмно поточні дату й час як рядок у текстовий файл today.txt. Зчитайте текстовий файл today.txt і розмістіть дані у рядку today_string. Розберіть дату з рядка today_string на складові, використовуючи функції для роботи з датом й часом, і виведіть їх на екран у вигляді повідомлень з пояснюючим текстом.
2. Створіть об'єкт happy, що містить дату вашого народження. Використайте функції для роботи з датою і часом, щоб дізнатися відповіді на такі питання (виведення організуйте в окремий файл у вигляді, на зразок «запитання: відповідь», у окремих рядках):
   a) "Яка дата вашого народження?"
   b) "У який день тижня ви народилися?"
   c) "Коли вам буде (або вже було) 13 330 днів від народження?"

3.3 Зміст звіту

Звіт повинен мати такі складові:

– титульний лист;
– мета роботи;
– результат виконання завдань;
– відповіді на контрольні питання.

3.4 Контрольні запитання

1. Відносно чого задається відносний шлях?
2. З чого починається абсолютний шлях?
3. Яке призначення функцій os.getcwd() і os.chdir()?
4. Що позначають імена . і ..?
5. Які частини шляху C:\Users\gt\logging\log12042019.txt представляють імена каталогів та ім’я файла?
6. Які функції використовуються для копіювання, перейменування і видалення файлів і каталогів?
7. Що таке «епоха Unix»?
8. Яка функція повертає кількість секунд, які пройшли з моменту початку «епохи Unix»?
9. У чому різниця між об’єктами datetime і timedelta?
10. Які інструменти локалізації присутні у Python?